

Zarr-Cesium: Client-side interactive 2D and 3D visualization of large multidimensional Zarr datasets in the browser

Tobias Ferreira ¹

¹ National Oceanography Centre, UK

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Environmental sciences increasingly rely on multidimensional datasets describing oceanic, atmospheric, and terrestrial systems across space, depth, and time. Numerical models and remote sensing products routinely produce terabyte-scale data cubes with dimensions such as longitude, latitude, elevation, and time (Eyring et al., 2016; Hersbach et al., 2020). These datasets are commonly stored in chunked, cloud-optimized formats such as Zarr (Zarr Developers, 2024), enabling scalable storage and parallel analysis.

Although storage and computational frameworks have evolved to support these multidimensional data structures (Abernathy et al., 2021; Zarr Developers, 2024), visualization workflows often transform them into two-dimensional raster products for web delivery, separating rendered outputs from the underlying data cube.

Zarr-Cesium is a TypeScript library that addresses these limitations. It enables interactive, browser-native visualization of multidimensional Zarr datasets within a 3D geospatial environment using CesiumJS (CesiumGS, 2024). The software streams chunked array data directly from object storage into the browser and renders it using WebGL2, without requiring backend services, tile-generation pipelines, or intermediate raster products.

The toolkit supports 2D scalar fields, 3D volumetric slicing, and animated vector fields. It is compatible with Zarr v2 and v3, multiscale pyramids (GeoZarr Developers, 2023; NDPyramid Developers, 2023), and CF-compliant metadata conventions (Eaton et al., 2025). By integrating cloud-native data access with GPU-accelerated rendering, Zarr-Cesium enables interactive exploration of large scientific datasets directly from their cloud-hosted storage.

Statement of Need

Although storage and computation frameworks have matured, visualization workflows remain fragmented across several partially overlapping ecosystems.

Scientific communication frequently relies on static figures or screenshots, which flatten inherently multidimensional systems into fixed two-dimensional views. Server-side tile-generation platforms convert datasets into raster products through Web Map Service (WMS) (Open Geospatial Consortium, 2006) or custom APIs, requiring persistent backend infrastructure, preprocessing steps, and operational maintenance. These workflows also duplicate data into derived tile products, increasing storage requirements and separating rendered output from its canonical source.

Three-dimensional visualization is frequently essential for scientific interpretation. Many environmental processes exhibit strong vertical structure, including ocean stratification and thermocline variability, mixed-layer dynamics, and large-scale overturning circulation (Pickard &

40 [Emery, 1990](#)). Atmospheric systems similarly depend on vertical structure, including boundary
41 layer evolution and convection ([Stull, 1988](#)). Horizontal maps alone can obscure depth-
42 dependent gradients, vertical shear, and subsurface anomalies that are central to physical
43 interpretation. Interpretation of velocity fields further requires contextualizing transport
44 pathways and circulation patterns across depth and time ([Marshall & Plumb, 1961](#)).

45 As multidimensional datasets become standard in operational modeling and reanalysis systems,
46 there is a need for visualization approaches that preserve their native structure rather than
47 flattening or duplicating them into derived products.

48 Statement of the Field

49 Existing visualization ecosystems address parts of this problem but do not provide a unified
50 client-side solution for multidimensional Zarr datasets.

51 Server-side tile-generation platforms convert multidimensional datasets into raster products
52 exposed through WMS endpoints or custom APIs. Systems such as GeoServer ([GeoServer
53 Developers, 2024](#)) and ncWMS ([Blower et al., 2013](#)) enable serving NetCDF-based datasets as
54 map layers but require persistent backend infrastructure and preprocessing steps to generate
55 tiles.

56 Cloud-native raster ecosystems based on Cloud-Optimized GeoTIFFs (COGs) ([Open Geospatial
57 Consortium, 2023](#)) enable efficient streaming of large two-dimensional imagery products. Tools
58 such as TiTiler ([Development Seed, 2023a](#)) provide scalable server-side generation of map
59 tiles directly from COGs. However, these approaches are fundamentally oriented toward two-
60 dimensional raster data. Extending them to higher-dimensional data cubes typically requires
61 precomputed slicing, separate endpoints for each non-spatial dimension, or additional backend
62 logic to manage depth and time selection.

63 For Zarr datasets, backend tile-generation solutions such as titiler-xarray ([Development Seed,
64 2023b](#)) and xpublish-tiles ([Earthmover, 2025](#)) expose Zarr-backed arrays through tile APIs.
65 While these tools allow dynamic slicing of multidimensional data, they still depend on server-side
66 infrastructure to perform array access, rasterization, and tile generation. Visualization therefore
67 remains mediated by backend services, and rendered products are transmitted as image tiles
68 rather than structured array subsets.

69 Client-side geospatial engines such as CesiumJS ([CesiumGS, 2024](#)) provide powerful 3D globe
70 visualization but do not natively support streaming and slicing of multidimensional chunked
71 array formats such as Zarr. Conversely, browser-based Zarr libraries such as Zarr.js ([Zarr.js
72 Developers, 2025](#)) enable direct access to chunked arrays in JavaScript but do not provide
73 geospatial integration, volumetric slicing, or GPU-based colormapping pipelines.

74 Consequently, no general-purpose framework currently supports direct streaming of
75 multidimensional Zarr arrays from object storage, automatic interpretation of CF-compliant
76 metadata, dynamic slice computation in the browser, GPU-based rendering of scalar and
77 vector fields, and fully static deployment without backend services. Zarr-Cesium fills this gap
78 by integrating these capabilities into a coherent client-side architecture.

79 Software Design

80 Zarr-Cesium is implemented in TypeScript and organized around three primary provider classes:
81 ZarrLayerProvider for 2D scalar fields, ZarrCubeProvider for 3D volumetric slicing, and
82 ZarrCubeVelocityProvider for animated vector fields. These providers encapsulate data
83 access, multidimensional slicing logic, and rendering integration.

84 All providers access Zarr datasets via a browser-based FetchStore, supporting HTTP range
85 requests, consolidated metadata, and both Zarr v2 and v3 specifications. Multiscale pyramid

86 datasets are detected through multiscales metadata, and resolution levels are selected
87 dynamically based on view scale. A lightweight in-memory cache maintains recently accessed
88 multiscale levels.

89 Dimension identification is performed automatically using CF standard_name attributes (Eaton
90 et al., 2025) and configurable alias mappings. Geographic bounds from Cesium tiles or
91 camera extents are translated into array index ranges, with automatic CRS detection between
92 EPSG:4326 and EPSG:3857. Zarr slice arguments are constructed dynamically, and value-based
93 selectors (e.g., timestamps or depths) are converted to nearest-index selections.

94 To maintain interactivity, the system limits concurrent chunk requests and uses
95 AbortController instances to cancel stale requests during rapid navigation. Only
96 visible spatial subsets are requested, and multiscale level selection reduces resolution at broader
97 zoom levels.

98 Rendering is performed using WebGL2. For scalar fields, float32 array subsets are uploaded
99 as R32F textures and processed in fragment shaders that apply scale-factor correction,
100 normalization, no-data masking, and dynamic colormapping. Because scaling and colormapping
101 occur on the GPU, visual parameters can be adjusted without re-fetching data.

102 For volumetric cubes, array subsets are reshaped into ndarray structures and horizontal or vertical
103 slices are generated dynamically. Textured Cesium Primitive geometries are constructed, and
104 elevation values are converted into globe-compatible heights with optional vertical exaggeration
105 or subsurface offsets. Slices are regenerated when dimension selectors or styling parameters
106 change.

107 Velocity fields are handled by loading U and V components from separate Zarr stores, extracting
108 elevation slices, and passing the data to the cesium-wind-layer library (Cesium Wind Layer
109 Developers, 2024) for particle-based animation. Multiple elevation layers can be rendered
110 simultaneously, enabling stacked three-dimensional flow visualizations.

111 By combining selective chunk loading, multiscale resolution management, GPU-based rendering,
112 and minimal data duplication, the architecture enables interactive exploration of datasets
113 significantly larger than browser memory limits. Performance is constrained primarily by chunk
114 size and network bandwidth rather than total dataset size.

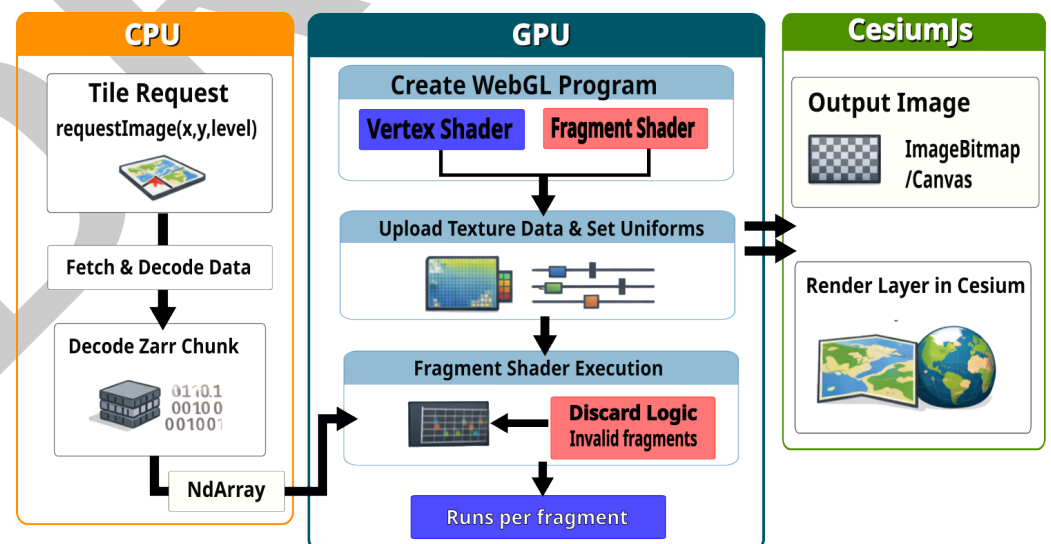


Figure 1: Representation of the workflow of the Zarr-Cesium package, from tile request to WebGL layer generation on the map.

Conclusion

Zarr-Cesium integrates cloud-native chunked storage access, CF-aware dimension resolution, multiscale pyramid handling, and GPU-based rendering into a unified client-side architecture.

The software does not introduce new rendering algorithms; rather, it formalizes a reusable and deployment-friendly approach for connecting multidimensional Zarr data directly to browser-based 3D environments. By rendering array subsets in the browser without intermediate rasterization or server-side preprocessing, Zarr-Cesium preserves fidelity to the canonical dataset stored in object storage and reduces ambiguity between analysis products and visualization artifacts.

Beyond infrastructure considerations, this architecture supports more transparent and exploratory analysis of inherently multidimensional environmental systems. Interactive slicing and dynamic rendering allow datasets to be examined in their full spatial and temporal structure rather than as predefined two-dimensional views. As cloud-native formats increasingly replace file-based distribution models, coupling scalable storage with browser-native visualization becomes an essential component of modern scientific infrastructure. Zarr-Cesium contributes such a component, supporting reproducible, interactive publication and communication of environmental data.

Acknowledgements

This work builds upon the open-source efforts of the CesiumJS, Zarrita, and cesium-wind-layer communities. Storage and testing infrastructure were supported by JASMIN, the UK's environmental data analysis facility. The project forms part of the AtlantiS initiative led by the National Oceanography Centre.

References

- Abernathy, R., Hamman, J., Lepore, C., & others. (2021). Cloud-native repositories for big scientific data. *Computing in Science & Engineering*, 23(2), 26–35. <https://doi.org/10.1109/MCSE.2021.3059437>
- Blower, J. D., Gemmell, A. L., Griffiths, G. H., Haines, K., Santokhee, A., & Yang, X. (2013). A web map service implementation for the visualization of multidimensional gridded environmental data. *Environmental Modelling & Software*, 47, 218–224. <https://doi.org/10.1016/j.envsoft.2013.04.002>
- Cesium Wind Layer Developers. (2024). *Cesium-wind-layer*. <https://github.com/hongfaqiu/cesium-wind-layer>
- CesiumGS. (2024). *CesiumJS*. <https://cesium.com/platform/cesiumjs/>
- Development Seed. (2023a). *TiTiler: A modern dynamic tile server for raster data*. <https://github.com/developmentseed/titiler>
- Development Seed. (2023b). *Titiler-xarray*. <https://github.com/developmentseed/titiler-xarray>
- Earthmover. (2025). *Xpublish-tiles*. <https://github.com/earth-mover/xpublish-tiles>
- Eaton, B., Gregory, J., Drach, B., Taylor, K., Hankin, S., Caron, J., Signell, R., Bentley, P., Rappa, G., Höck, H., Pamment, A., Jukes, M., Raspaud, M., Blower, J., Horne, R., Whiteaker, T., Blodgett, D., Zender, C., Lee, D., ... Laake, P. V. (2025). *NetCDF climate and forecast (CF) metadata conventions* (Version 1.13). CF Community. <https://doi.org/10.5281/zenodo.17801666>

- 158 Eyering, V., Bony, S., Meehl, G. A., Senior, C. A., Stevens, B., Stouffer, R. J., & Taylor,
159 K. E. (2016). Overview of the coupled model intercomparison project phase 6 (CMIP6).
160 *Geoscientific Model Development*, 9, 1937–1958. [https://doi.org/10.5194/gmd-9-1937-](https://doi.org/10.5194/gmd-9-1937-2016)
161 [2016](https://doi.org/10.5194/gmd-9-1937-2016)
- 162 GeoServer Developers. (2024). *GeoServer*. <https://geoserver.org>
- 163 GeoZarr Developers. (2023). *GeoZarr specification*. [https://github.com/zarr-developers/](https://github.com/zarr-developers/geozarr-spec)
164 [geozarr-spec](https://github.com/zarr-developers/geozarr-spec)
- 165 Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horanyi, A., Muñoz-Sabater, J., Nicolas,
166 J., Peubey, C., Radu, R., Schepers, D., & others. (2020). The ERA5 global reanalysis.
167 *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049. [https:](https://doi.org/10.1002/qj.3803)
168 [/doi.org/10.1002/qj.3803](https://doi.org/10.1002/qj.3803)
- 169 Marshall, J., & Plumb, R. A. (1961). *Atmosphere, ocean and climate dynamics: An introductory*
170 *text* (Vol. 2). Academic Press.
- 171 Numpy Developers. (2023). *Numpy: Multiscale pyramids for multidimensional arrays*.
172 <https://github.com/carbonplan/ndpyramid>
- 173 Open Geospatial Consortium. (2006). *Web map service (WMS) implementation specification*.
174 <https://www.ogc.org/standards/wms>
- 175 Open Geospatial Consortium. (2023). *OGC Cloud Optimized GeoTIFF Standard* (J. Masó, Ed.;
176 OGC Implementation Standard No. 21-026; Version 1.0). Open Geospatial Consortium.
177 <https://docs.ogc.org/is/21-026/21-026.html>
- 178 Pickard, G. L., & Emery, W. (1990). *Descriptive physical oceanography: An introduction* (p.
179 320). ISBN: 075062759X
- 180 Stull, R. B. (1988). *An introduction to boundary layer meteorology*. Kluwer Academic
181 Publishers. <https://doi.org/10.1007/978-94-009-3027-8>
- 182 Zarr Developers. (2024). *Zarr: Open standard for chunked, compressed n-dimensional arrays*.
183 <https://zarr.dev/>
- 184 Zarr Developers. (2025). *Zarr: A minimal & modular zarr implementation in TypeScript*.
185 <https://github.com/manzt/zarr.js>